

Automatisation Acquisition Client

LinkedIn & Instagram

Guide technique de deployment

n8n + Claude AI + Playwright + PostgreSQL

Systeme production-ready, auto-heberge, sans licence payante

Version	1.0
Date	Mai 2026
Environnement	Docker Compose
IA	Claude (Anthropic)

Table des matieres

1. Architecture du systeme
 2. Configuration Docker (docker-compose.yml)
 3. Serveur Playwright
 4. Base de donnees (init-db.sql)
 5. Workflows n8n (5 workflows)
 6. Prompts Claude (6 prompts)
 7. Scripts utilitaires
 8. Documentation de deploiement
 9. Gestion des risques et conformite
 10. Checklist production-ready
- Annexe A : Structure du projet
- Annexe B : Variables d'environnement

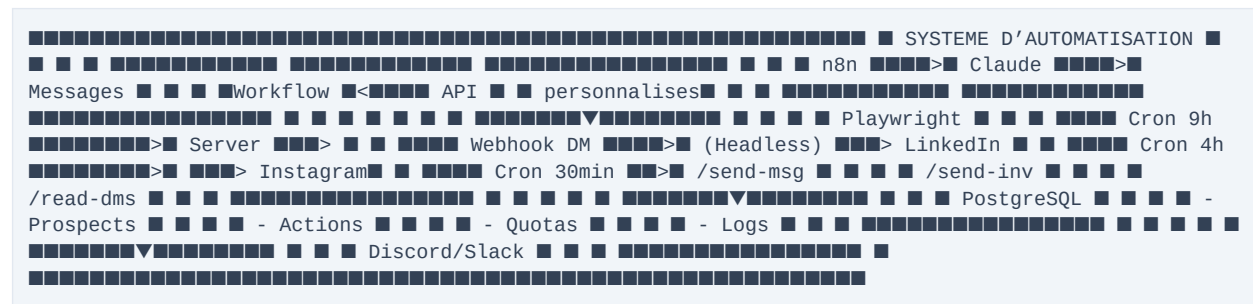
1. Architecture du systeme

Ce système automatise l'acquisition client sur LinkedIn et Instagram. Il est composé de 4 composants principaux orchestrés par n8n, avec Claude AI pour l'intelligence conversationnelle et Playwright pour les actions navigateur.

1.1 Composants

Composant	Rôle	Technologie
n8n	Orchestrateur de workflows	Node.js, UI web
PostgreSQL	Stockage des données	PostgreSQL 16
Playwright Server	Actions navigateur	Node.js, Chromium
Claude API	Intelligence conversationnelle	Anthropic API
Discord/Slack	Alertes et notifications	Webhooks

1.2 Schema d'architecture



1.3 Flux de donnees - Outreach

Le flux principal d'outreach suit 5 étapes, depuis l'identification du prospect jusqu'à la conversion. Chaque étape est tracée dans la base de données et soumise à des contrôles de quota.

Etape	Action	Service
1	Identification prospects (score, fit)	PostgreSQL + Claude
2	Generation message personnalise	Claude API
3	Envoi connexion / DM	Playwright Server
4	Suivi et relance (sequence)	n8n Workflow
5	Reponse automatique DMs	Claude + Playwright
6	Alerte lead chaud	Discord Webhook

1.4 Securisation

- Tous les secrets dans le fichier .env (jamais dans le code)
- Réseau Docker interne isole (automation-net)
- Cle API pour securiser le serveur Playwright (header x-api-key)

- Chiffrement AES-256 des identifiants stockés en base
- Authentification basique HTTP pour n8n
- Mode DRY_RUN pour tester sans action réelle

2. Configuration Docker

L'ensemble du système est déployé via un seul fichier `docker-compose.yml` avec 3 services actifs et 1 optionnel. Chaque service est configuré avec des volumes persistants, des healthchecks, et un redéploiement automatique.

2.1 Services

Service	Image	Port	Volume
PostgreSQL	postgres:16-alpine	5433:5432	postgres_data
n8n	n8nio/n8n:latest	5678:5678	n8n_data
Playwright	build: ./playwright-server	3100:3000	playwright_logs
Redis (optionnel)	redis:7-alpine	6380:6379	redis_data

2.2 Extrait du `docker-compose.yml`

```
services: postgres: image: postgres:16-alpine container_name: automation-postgres restart:
unless-stopped ports: ["5433:5432"] volumes: - postgres_data:/var/lib/postgresql/data -
./init-db.sql:/docker-entrypoint-initdb.d/init-db.sql:ro healthcheck: test: ["CMD-SHELL", "pg_isready
-U automation"] n8n: image: n8nio/n8n:latest container_name: automation-n8n ports: ["5678:5678"]
environment: DB_TYPE: postgresdb DB_POSTGRESDB_HOST: postgres N8N_BASIC_AUTH_ACTIVE: "true" volumes:
[n8n_data:/home/node/.n8n] depends on: postgres: { condition: service_healthy }
```

Fichier complet : `automation-project/docker-compose.yml`

3. Serveur Playwright

Le serveur Playwright est un serveur Express.js qui expose des endpoints HTTP securises pour piloter un navigateur Chromium headless. Il gere automatiquement les sessions de connexion, le rate limiting, et les quotas quotidiens. Chaque action est journalisee en format JSON structure.

3.1 Endpoints

Methode	Endpoint	Description
GET	/health	Healthcheck du service
POST	/send-invite	Invitation LinkedIn
POST	/send-message	DM LinkedIn ou Instagram
POST	/read-messages	Lire les DMs recus
POST	/accept-invite	Accepter une invitation
POST	/scrape-profile	Extraire profil
POST	/batch-action	Actions multiples

3.2 Gestion des quotas

Plateforme	Action	Limite/jour	Delai min
LinkedIn	Invitations	80	45 secondes
LinkedIn	Messages DM	150	45 secondes
Instagram	Messages DM	50	60 secondes

3.3 Mode DRY_RUN

En positionnant DRY_RUN=true dans le .env, le serveur ne lance aucun navigateur. Il simule chaque action et retourne un resultat fictif. C'est indispensable pour tester le pipeline complet sans risque de bannissement.

Fichier source : [automation-project/playwright-server/playwright-server.js](#) (2081 lignes)

4. Base de donnees

Le schema PostgreSQL comprend 7 tables couvrant l'ensemble du cycle de vie d'un prospect. Le fichier init-db.sql est automatiquement execute au premier demarrage de PostgreSQL via Docker.

4.1 Tables principales

Table	Role	Cles
prospects	Profil et statut	status, score, sequence_id
sequences	Plans de suivi multi-etapes	steps (JSONB)
conversations	Historique des echanges	claude_analysis (JSONB)
actions	Journal des actions	status, executed_at
daily_quotas	Compteurs de quotas	UNIQUE(platform, date)
account_credentials	Identifiants chiffrés	encrypted_password
automation_logs	Logs structures	level, service

4.2 Cycle de vie d'un prospect

Statuts : new -> contacted -> replied -> qualified -> converted -> rejected / unsubscribed
Transitions automatisees par les workflows n8n : new : Prospect identifie, pas encore contacte
contacted : Invitation envoyee ou premier DM replied : Le prospect a repondu qualified : Score >= 7,
lead chaud converted : Deal signe ou meeting obtenu

Fichier source : automation-project/init-db.sql

5. Workflows n8n

Les 5 workflows couvrent l'ensemble du cycle d'automatisation. Chaque workflow est un fichier JSON importable dans n8n. Ils sont inactifs par défaut (active: false) et doivent être activés manuellement.

5.1 Workflow 1 - Outreach sequentiel

Declenche chaque jour a 9h. Recupere 20 prospects (score desc, statut new), genere un message personnalise via Claude, envoie les invitations une par une avec un delai de 45-120s entre chaque action. Se met en pause si le quota est atteint.

Noeud	Type	Action
Cron Trigger	Planificateur	Quotidien 9h00
PostgreSQL	Lecture	SELECT prospects new LIMIT 20
Claude API	HTTP Request	Message personnalise
Code	JS	Jitter 45-120s
Playwright	HTTP Request	POST /send-invite
PostgreSQL	Ecriture	UPDATE status
Discord	Webhook	Alerte erreur

5.2 Workflow 2 - Reponse auto DMs

Declenche par webhook (POST /webhook/dm-received). Analyse le DM entrant avec Claude, determine l'intention et le score (0-10). Hot (≥ 7) -> reponse + alerte Discord ; Warm (3-6) -> reponse douce ; Cold (< 3) -> log seul.

5.3 Workflow 3 - Creation de contenu

Lundi et jeudi a 10h. Genere un post LinkedIn et une legende Instagram via Claude, sauvegarde en base, attend 2h pour verification, puis publie.

5.4 Workflow 4 - Gestion des sequences

Toutes les 4h. Recupere les prospects en attente de relance, calcule la prochaine etape (max 5), genere le message via Claude, envoie via Playwright, verifie si reponse.

5.5 Workflow 5 - Monitoring

Toutes les 30min. Verifie en parallele : (A) sante Playwright, (B) quotas, (C) taux d'erreur. Genere un resume quotidien a minuit.

Fichiers sources : automation-project/workflows/ (5 fichiers JSON)

6. Prompts Claude

Les 6 prompts sont utilises dans les workflows n8n. Chaque prompt a un template system et user avec des variables {{variable}} remplacees dynamiquement.

6.1 Liste des prompts

ID	Nom	Usage
A	OUTREACH_GENERATION	Message de connexion personnalise
B	INCOMING_DM_ANALYSIS	Analyse DM entrant (score + intent)
C	CONTEXTUAL_AUTO_REPLY	Reponse automatique contextuelle
D	LINKEDIN_POST_CREATION	Post LinkedIn professionnel
E	PROSPECT_QUALIFICATION	Qualification profil prospect
F	DAILY_SUMMARY	Resume quotidien

6.2 Exemple - Prompt A (Outreach)

System : Tu es un assistant commercial B2B. Tu rediges des messages de connexion LinkedIn personnalises et concis. Tu ne dois jamais paraitre commercial ou insistant. Reference un element specifique du profil. Maximum 300 caracteres. Ton professionnel et chaleureux, en francais.

User : Genere un message de connexion LinkedIn pour {{name}}, {{role}} chez {{company}}. Son profil mentionne : {{headline}}. Contexte : {{extra_context}}.

6.3 Fonctions utilitaires

```
const message = renderPrompt('OUTREACH_GENERATION', { name: 'Marie Dupont', role: 'CTO', company: 'TechCorp', headline: 'Data et IA' }); const req = buildClaudeRequest('INCOMING_DM_ANALYSIS', { platform: 'linkedin', sender name: 'Marie', message: 'Votre solution m'interesse...' });
```

Fichier source : automation-project/prompts/claude-prompts.js

7. Scripts utilitaires

7.1 deploy.sh

Vérifie les prérequis, génère le .env, construit les images Docker, démarre les services, attend les healthchecks.

```
chmod +x scripts/deploy.sh ./scripts/deploy.sh # Resultat : n8n:5678, Playwright:3100, PG:5433
```

7.2 backup.sh

Dump PostgreSQL, copie les volumes, compresse en tar.gz, conserve les 7 dernières sauvegardes.

```
# Sauvegarde manuelle ./scripts/backup.sh # Cron quotidien à 3h du matin 0 3 * * *  
/chemin/scripts/backup.sh
```

7.3 health-check.sh

Vérifie saine services, espace disque, mémoire. Sort un rapport JSON.

7.4 utils.js

Module CommonJS avec 9 fonctions : randomDelay, checkQuota, sendDiscordAlert, sendSlackAlert, dryRunLog, getBackoffDelay, isValidProfileUrl, maskSensitive, getOptimalSendTime.

7.5 quota-manager.js

CLI + module PostgreSQL. Commandes : report, reset, check, increment.

```
node scripts/quota-manager.js report node scripts/quota-manager.js check --platform=linkedin  
--action=invite node scripts/quota-manager.js reset
```

Fichiers sources : automation-project/scripts/ (5 fichiers)

8. Documentation de deployment

8.1 Prerequis

- Docker et Docker Compose (v2.x minimum)
- Node.js 18+ (optionnel, pour scripts hors Docker)
- Cle API Anthropic (Claude)
- Compte LinkedIn avec session active
- Compte Instagram avec session active
- Webhook Discord ou Slack pour alertes
- Serveur ou VPS avec 4 Go RAM minimum

8.2 Installation pas a pas

Etape 1 : Cloner et acceder au dossier

```
cd automation-project
```

Etape 2 : Configurer les variables d'environnement

```
cp .env.example .env # Remplacer TOUTES les valeurs CHANGEZ MOI
```

IMPORTANT : Ne jamais commiter le fichier .env dans Git.

Etape 3 : Lancer le deployment

```
chmod +x scripts/deploy.sh ./scripts/deploy.sh
```

Etape 4 : Initialiser la base (si necessaire)

```
docker exec -i automation-postgres psql \ -U automation -d automation < init-db.sql
```

Etape 5 : Importer les workflows dans n8n

Ouvrir <http://localhost:5678>, se connecter, puis importer les 5 fichiers JSON depuis workflows/.

Etape 6 : Tester en DRY_RUN

Positionner DRY_RUN=true dans .env et redemarrer Playwright. Lancer le workflow d'outreach et verifier les logs.

Etape 7 : Passer en production

DRY_RUN=false, activer les workflows un par un. Tester avec un compte sandbox d'abord.

9. Gestion des risques et conformite

9.1 Limites des plateformes

Limite	LinkedIn	Instagram
Invitations/jour	80 (recommande)	N/A
Messages DM/jour	150	50
Delai entre actions	45s minimum	60s minimum
Jitter recommande	45-120s	60-120s
Heures actives	8h-20h Europe	9h-21h Europe

9.2 Strategies anti-bannissement

- **Mode veille** : si 3 erreurs 429 en 10 min, pause automatique 2h
- **Backoff exponentiel** : delai progressif apres chaque erreur (max 5 min)
- **Rotation de comptes** : identifiants chiffres en base, basculement entre comptes SDR
- **Horaires restrictifs** : actions limitees 8h-20h, reduites le week-end
- **DRY_RUN** : tester sans risque

9.3 Journalisation pour audit

Chaque action est journalisee dans automation_logs (niveau, service, metadonnees). Les erreurs sont dans la table actions avec le message detaille. Cela permet un audit complet et la reconstitution de la chronologie.

9.4 Stockage des identifiants

Les identifiants sont chiffres avec AES-256 avant stockage en base. La cle ENCRYPTION_KEY est fournie via .env. En cas de compromission du serveur, les identifiants restent proteges.

10. Checklist production-ready

Utilisez cette checklist avant le passage en production (DRY_RUN=false).

- [] **Securite** : Secrets dans .env (jamais dans le code)
- [] **Securite** : .env dans .gitignore
- [] **Securite** : Identifiants chiffres AES-256 en base
- [] **Securite** : x-api-key requis pour Playwright
- [] **Securite** : Auth basique HTTP pour n8n
- [] **Donnees** : Backup PostgreSQL quotidien (cron)
- [] **Donnees** : Volumes Docker persistants
- [] **Donnees** : Script de restauration teste
- [] **n8n** : 5 workflows importes et actifs
- [] **n8n** : Variables accessibles via {{\$env.NOM}}
- [] **n8n** : Monitoring toutes les 30 minutes
- [] **Playwright** : Restart automatique apres crash
- [] **Playwright** : Healthcheck /health = 200
- [] **Playwright** : Sessions cookies persistees
- [] **Playwright** : DRY_RUN teste avec succes
- [] **Resilience** : Backoff exponentiel actif
- [] **Resilience** : Mode veille auto apres 3 erreurs 429
- [] **Monitoring** : Alertes Discord fonctionnelles
- [] **Monitoring** : health-check.sh en cron
- [] **Monitoring** : Logs centralises
- [] **Production** : Test complet avec compte sandbox
- [] **Production** : Quotas configures correctement

Annexe A : Structure du projet

```
automation-project/ docker-compose.yml # Configuration Docker .env.example # Template variables env
init-db.sql # Schema PostgreSQL (7 tables) playwright-server/ Dockerfile # Image Docker multi-stage
playwright-server.js # Serveur Express (2081 lignes) workflows/ 1-outreach-sequentiel.json
2-reponse-auto-dms.json 3-creation-contenu.json 4-gestion-sequences.json 5-monitoring-alertes.json
prompts/ claude-prompts.js # 6 prompts + fonctions scripts/ deploy.sh backup.sh health-check.sh
utils.js quota-manager.js
```

Annexe B : Variables d’environnement

Variable	Requise	Defaut
POSTGRES_PASSWORD	Oui	-
N8N_BASIC_AUTH_USER	Oui	admin
N8N_BASIC_AUTH_PASSWORD	Oui	-
ANTHROPIC_API_KEY	Oui	-
PLAYWRIGHT_API_KEY	Oui	-
DISCORD_WEBHOOK_URL	Recommande	-
CLAUDE_MODEL	Non	claude-sonnet-4-20250514
LINKEDIN_MAX_INVITATIONS_PER_DAY	Non	80
LINKEDIN_MIN_DELAY_MS	Non	45000
DRY_RUN	Non	false
LOG_LEVEL	Non	info
ENCRYPTION_KEY	Oui	-